

Day 10: Docker Compose & Web App Hosting

Multi-Container Orchestration & CarePlus Hospital Case Study

Module 1: Docker Compose Fundamentals

Docker Compose is a tool developed to define, share, and run multi-container applications on a single host. With Docker Compose, you use a YAML configuration file to specify your application's services, networks, and storage volumes. This replaces long, manual command-line execution sequences where dependencies between containers are difficult to configure.

- **Single-Command Control:** Instead of typing multiple docker commands, developers can launch, build, update, and shut down their entire infrastructure using a single command: `docker compose`.
- **Service Isolation & Links:** Compose automatically sets up a default network linking all defined services. Containers can communicate with one another securely using their service names as hostname aliases.
- **Declarative Configurations:** All configurations are written in a plain `compose.yml` file, allowing deployment parameters to be version-tracked in Git alongside application code.

Core Docker Compose CLI Commands

Command	Primary Action	Usage Context
<code>docker compose up</code>	Creates, starts, and links services defined in the configuration	Run ' <code>docker compose up -d</code> ' to start services in background daemon mode.
<code>docker compose down</code>	Stops and removes containers, networks, and volumes	Cleans up local running instances and frees up host resources.
<code>docker compose ps</code>	Lists running containers managed under the current project	Check container IDs, active ports, and container health states.
<code>docker compose logs</code>	Aggregates and streams standard output logs from services	Use ' <code>docker compose logs -f</code> ' to follow log streams in real-time.
<code>docker compose build</code>	Forces a rebuild of container images specified with a build tag	Execute this after updating code, static files, or Dockerfiles.
<code>docker compose exec</code>	Executes an interactive shell or script inside a container	Example: ' <code>docker compose exec web bash</code> ' opens a terminal inside web service.

Module 2: Hosting the CarePlus Hospital Web App

In this module, we examine how to host the CarePlus Hospital web application (a Python Flask-based portal) using Docker Compose. The CarePlus system features sections for Core Medical Services (Cardiology, Neurology, Orthopedics, General Medicine), an Interactive Health Hub (BMI Calculator & Triage Symptom Bot), and an online Appointment Booking system. The deployment is fully orchestrated using three key files: app.py, Dockerfile, and compose.yml.

1. Application Source Code: app.py

The Python script utilizes the Flask framework to define web routing paths, process form submissions for appointments, calculate BMI dynamically, and return responses.

```
from flask import Flask, render_template, request, jsonify
app = Flask(__name__)

@app.route('/')
def home():
    return 'CarePlus Hospital Portal - Active'

@app.route('/calculate_bmi', methods=['POST'])
def calculate_bmi():
    data = request.get_json()
    weight = float(data.get('weight', 0))
    height = float(data.get('height', 0)) / 100
    bmi = weight / (height ** 2) if height > 0 else 0
    return jsonify({'bmi': round(bmi, 2)})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

2. Container Build Blueprint: Dockerfile

The Dockerfile compiles the source code into an immutable image, pulling a light python base runtime, installing Flask libraries, copying files, exposing port 5000, and launching the application server.

```
FROM python:3.11-slim
WORKDIR /app
RUN pip install --no-cache-dir flask
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Module 2: Hosting CarePlus (Continued)

3. Service Orchestration: compose.yml

The compose.yml YAML file tells Docker how to run the CarePlus application. It defines a service named 'careplus_app', maps external port 5000 to internal port 5000, mounts the local file path as a volume (allowing live updates to app.py without rebuilding the image), and sets restart rules.

```
services:
  careplus_app:
    build: .
    container_name: careplus_web
    ports:
      - "5000:5000"
    volumes:
      - ./app
    restart: always
```

How to Host and Deploy the Web App

- **Directory Setup:** Ensure all three files are saved in the same directory (e.g. /home/ubuntu/careplus/).
- **Build and Run:** Execute the following command in Git Bash to build the image and run the container in the background:
docker compose up -d
- **Verify Port Mapping:** Confirm the container is running and listening on port 5000 using 'docker compose ps'. Check the system logs with 'docker compose logs'.
- **Accessing Web Site:** Open your browser and navigate to 'http://<your-ec2-public-ip>:5000'. The CarePlus Hospital services, BMI tool, and appointment form will load instantly.